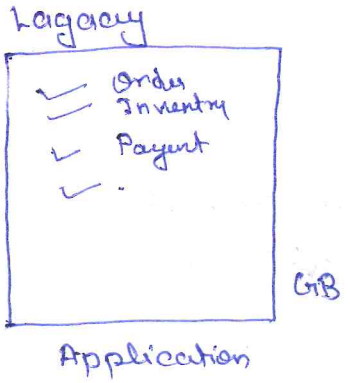
MicroServices Design Pattern

Monolethic & Microservices



Online Store → Disadvantages

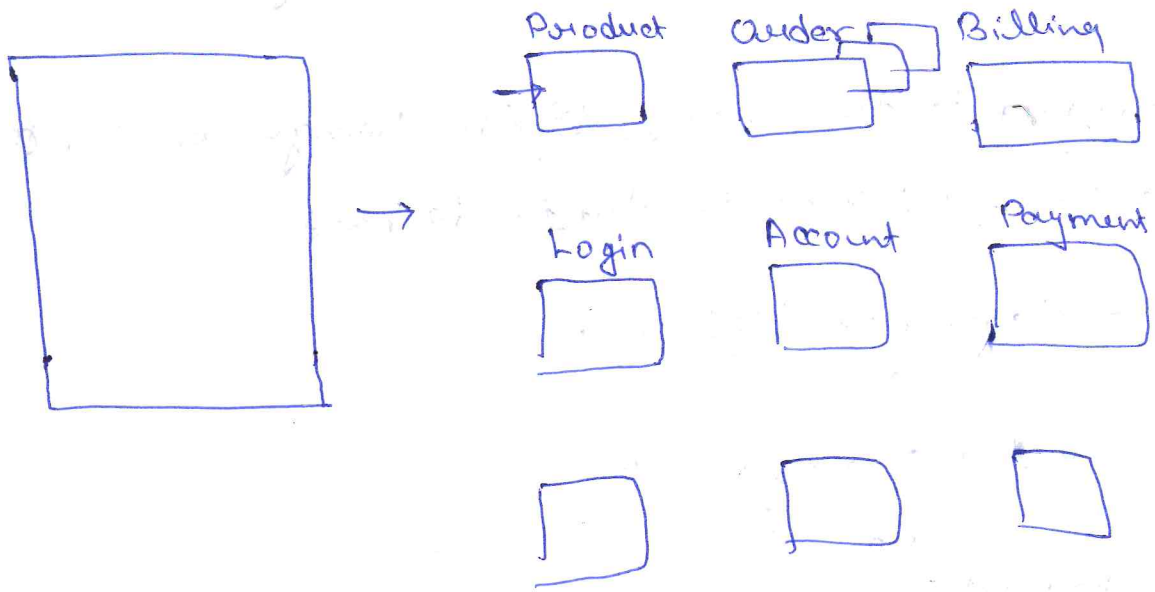
- Overload IDE
- Scaling is very hard
  - → CI job fast

2 * If sub Operation (CI job fast) is ~~easy~~ fast then easy to scaling.

3 :- Monoletic is Tight coupled. If we change one line it might Impact so much domain.

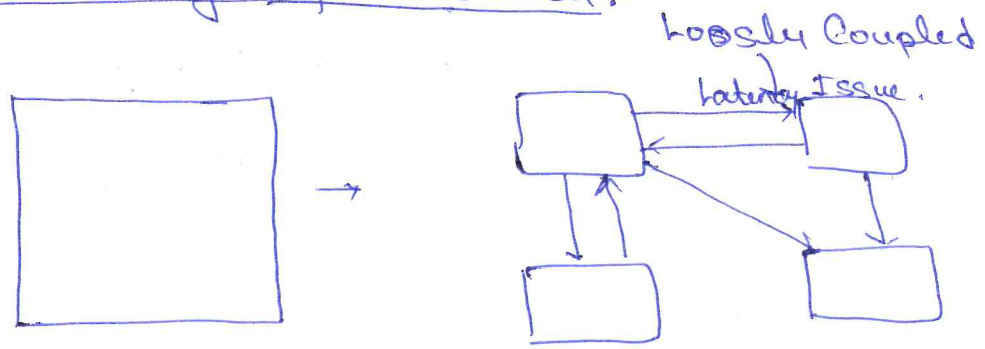
Concept && Coding

Why Microservices ??



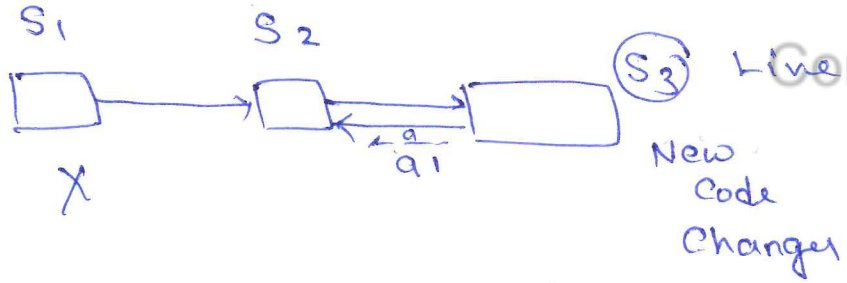
1- Managing is very easy to developer
2- We can scale Impendetly.

Disadvantages of Microservices :-



① We can proper break the ~~non~~-microservices. All Component are loosly coupled. All component are tightly coupled we give more communication. It a latency Issue.

②

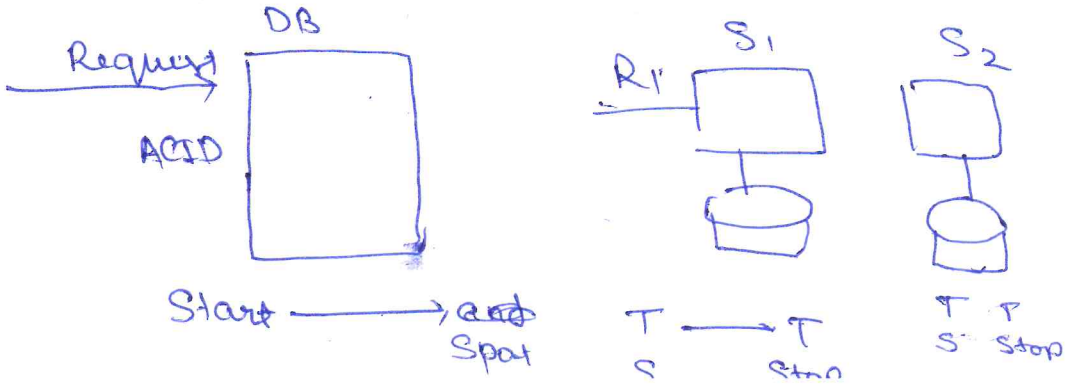


Concept && Coding

Debuging is two hard. If the changes on S_3. It can find who is clupit is hard.

* Monitering is diffecult.

③ Transtion Management :-



MicroServices

Micro — Services

Small — How much small ??

Phases of MicroServices :-

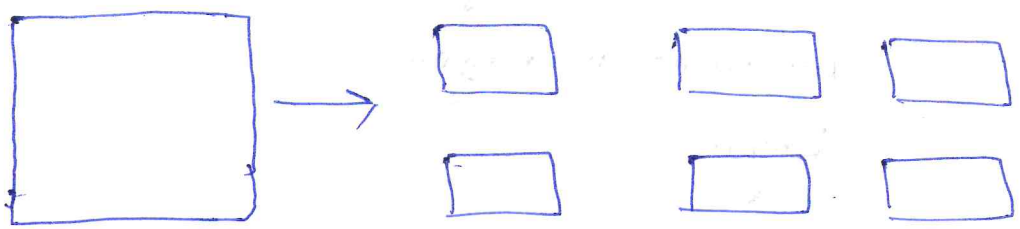
1 :- Decomposition	2 : Database	3 Communication
• Decompose by Bussiness capability • Decompose by Subdomain	• Database per Service • Database Shared	API Event

4 : Integration → Observelity, Monitering

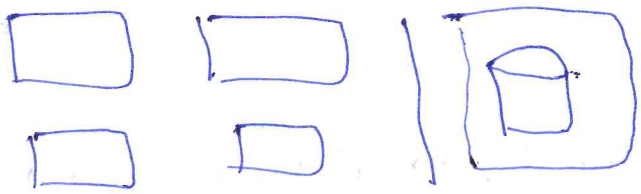
API Gateway

Concept && Coding

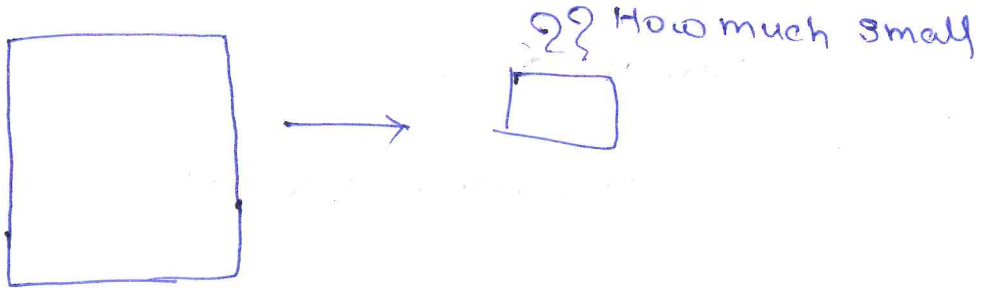
1 : Decomposition :-



2 :- Database :-



1:. Decomption Pattern :-



By Bussiness fuctionility / Capability

By Subdomain DDD — Domain Driven Design

1. Bussiness Capability :-

Concept && Coding

Online OrderApplication

ORDER Managemnt

PRODUCT Managmnt

Account Managmnt

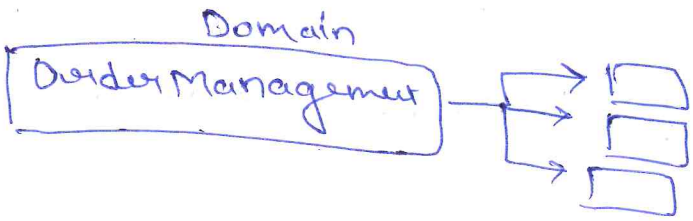
Login

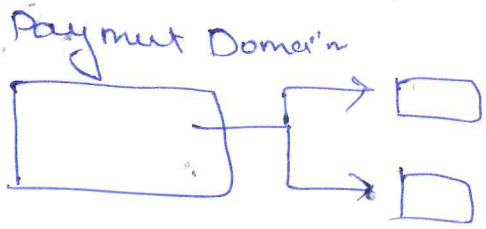
Challage :-

Knowledge of ~~what~~ which function on your Bussiness.

* There is no defention How much Micro.

2: By Subdomain (DDD) :-

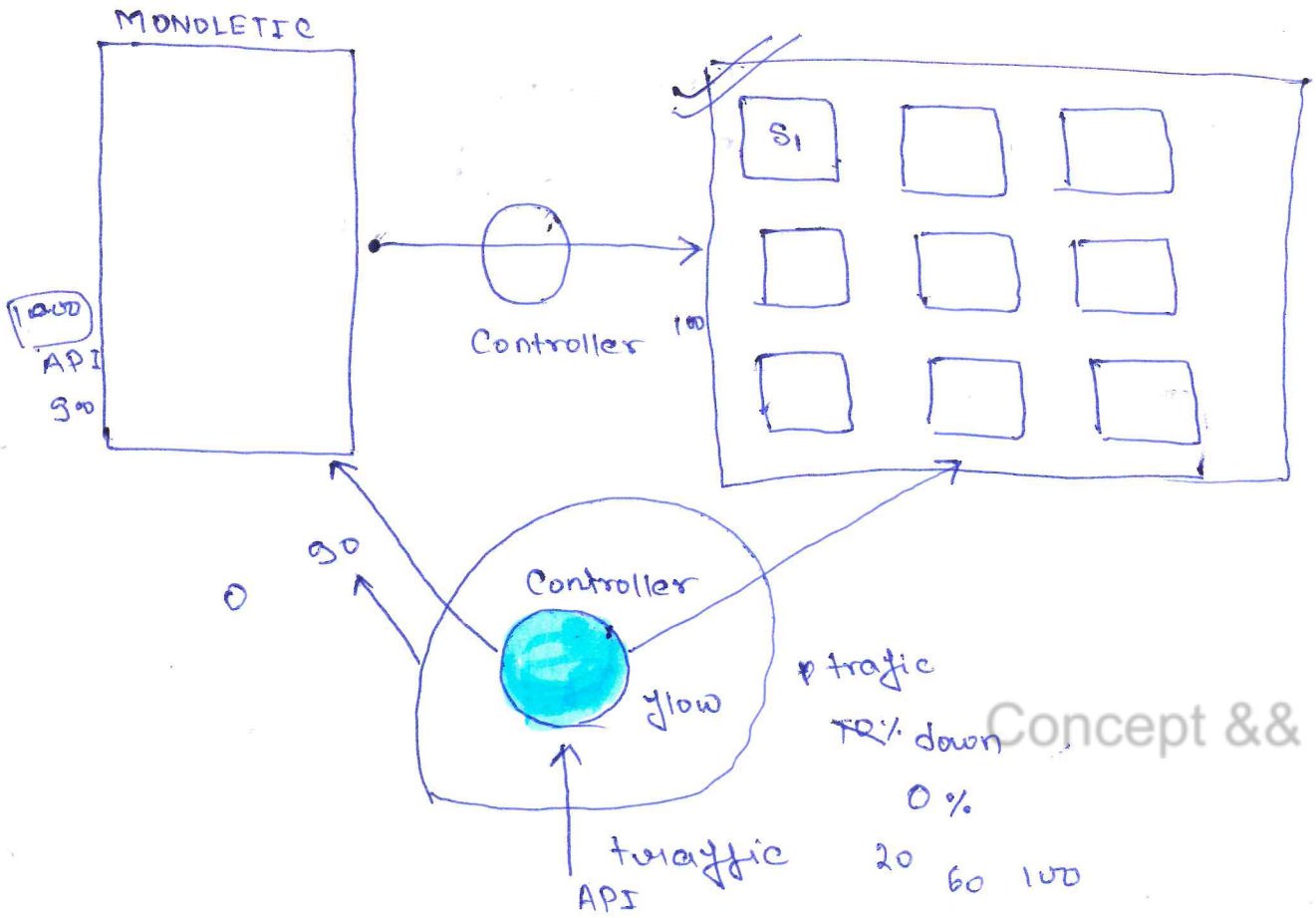




Concept && Coding

HLD SAGA Pattern | Strangler Pattern | CQRS |

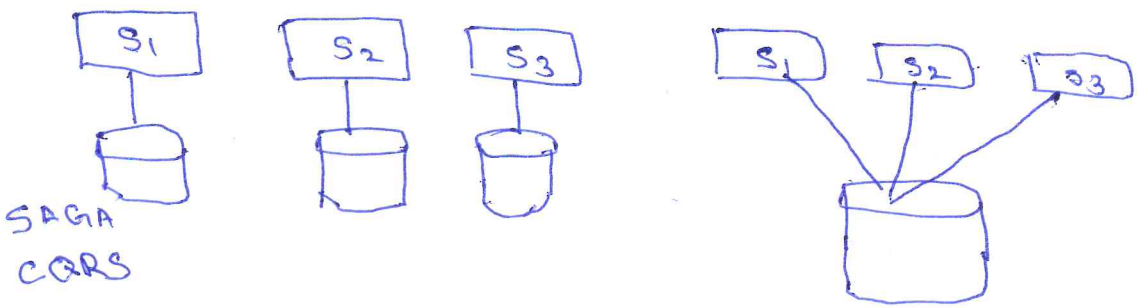
Strangler Pattern :- (Refactoring to Microservices)



Concept && Coding

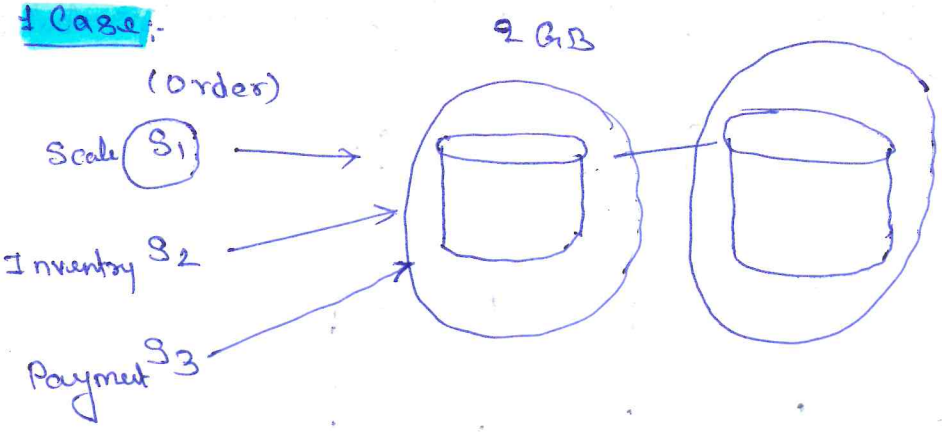
* Data Management in Microservices

* Database for Each Individual Service
* Shared Database



Why Shared database is not Sucess :-

1 Case :-

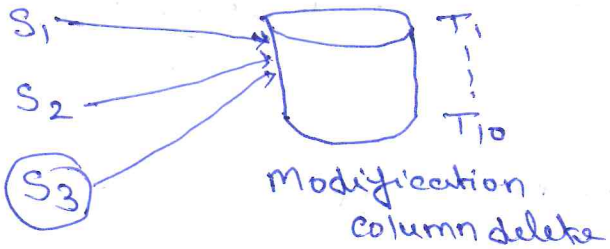


Suppose order have lots of traffic If we try to scale only order service that moment, we have scale the whole database

1 We can't scale independently.

2 Case :-

Concept && Coding



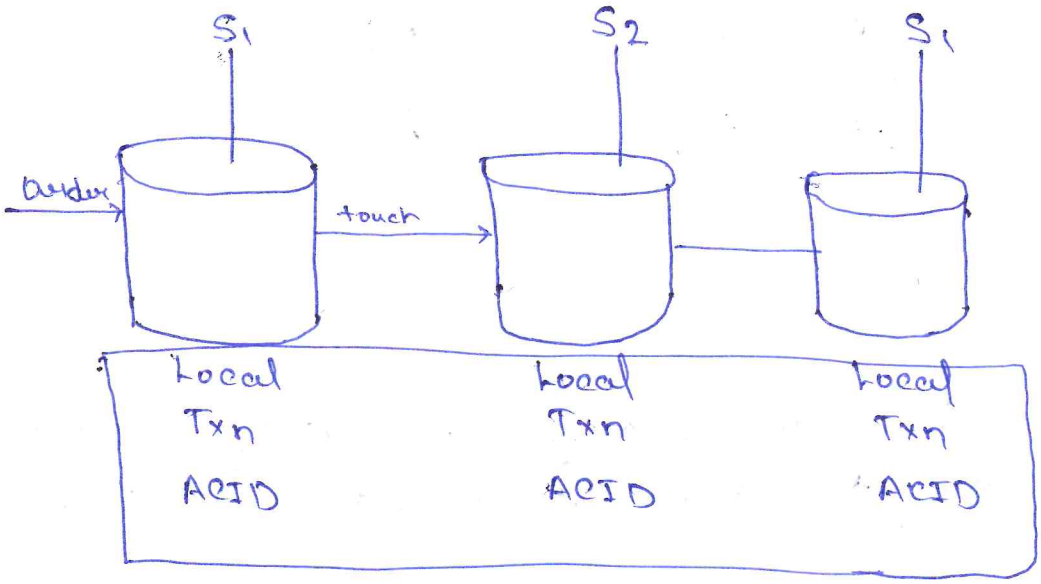
Suppose that one team have common database. There are 10 table in DB. If S3 want to some modification in table 10. S3 want to delete a one column. S3 will not delete the column. Firstly check dependency.

Easy

- Query Join, (10 Tabale Easy to join) : CQRS have to solve DD
- Easy to Maintain the transition property. (ACID)

~~Note:- That is~~

Note:- Shared Database easy task is difficult in Distributed Database.



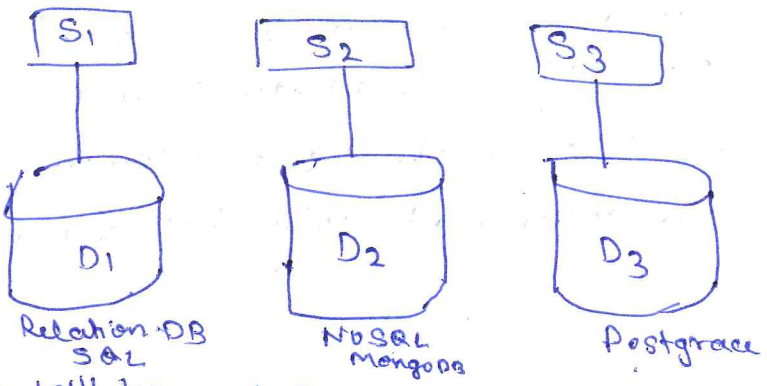
Now my transition is spaning over the Distributed Database.

Challange.

* This chanllange is solve to SAGA Design Pattern

Concept && Coding

Database-per-Services



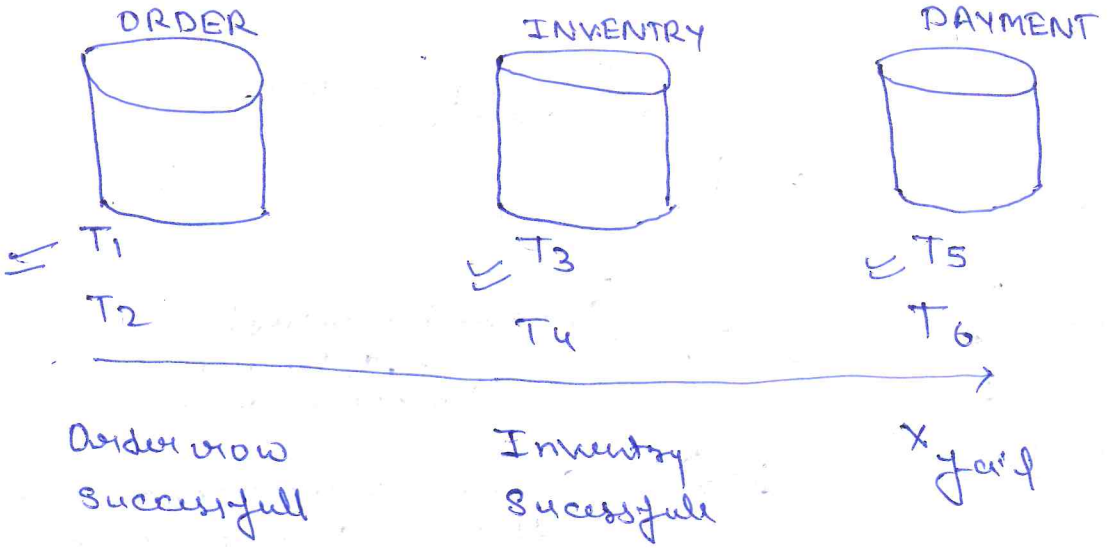
- S_2 will be not Query on DB_3. ~~If want to~~ If S_2 want to DB_3 Data, S_2 will be firstly request to S_3 through API. S_3 have give the data. filly S_2 will be.

- Maintain to easy a particalar DB
- We have scale independly.

SAGA

Why we need Saga ??

Txn ⇒ Place an Order - Payment



* ACID Property tell that If sucess will be all or none.

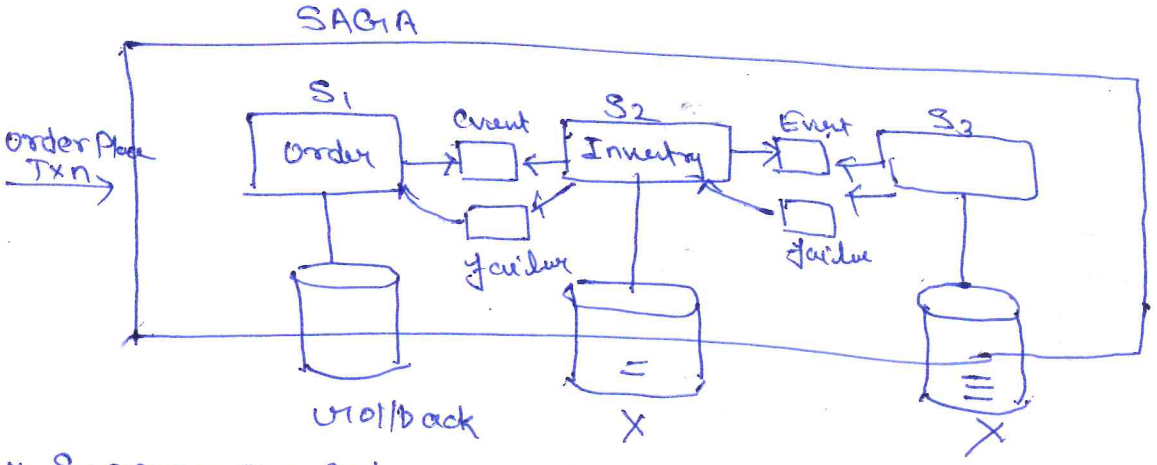
Local ACID ?

Local Txn ?

Concept && Coding

That's need a Saga ??

- Saga means sequence of Local Transition.

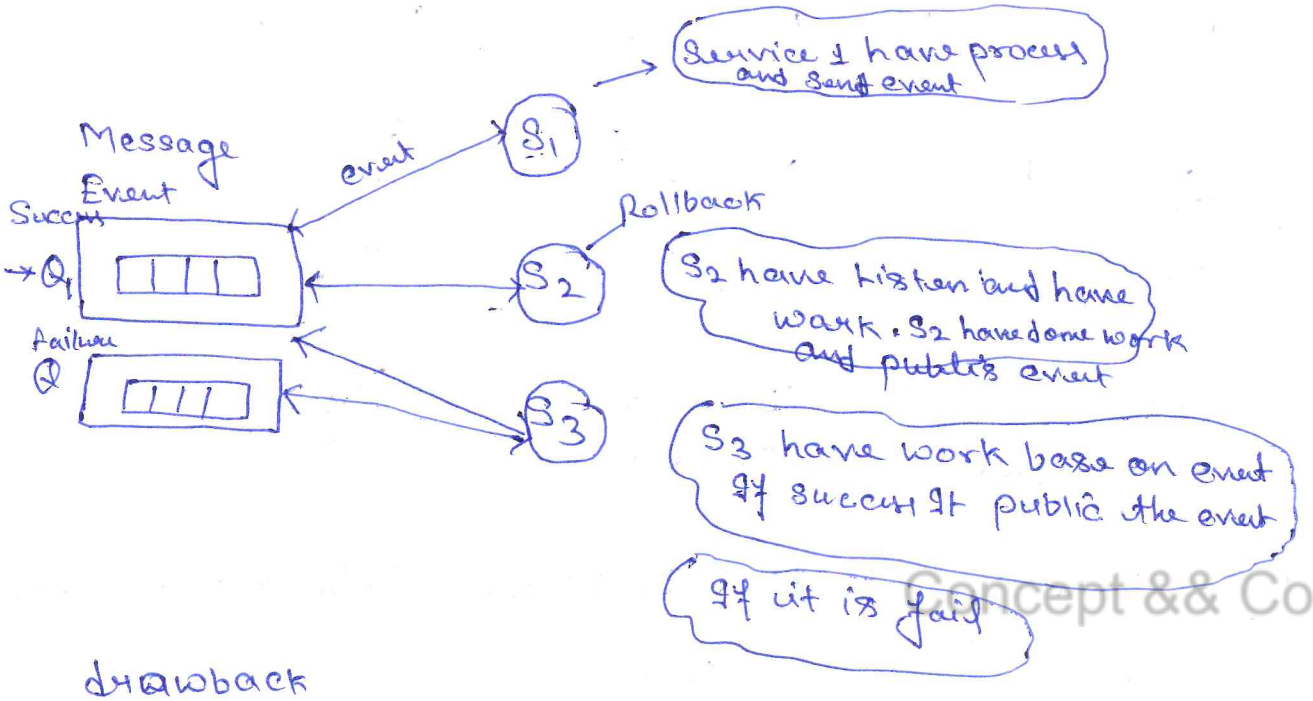


* Suppose one Order request come Order have update on DB and publish a one event. Inventry have Listen the Order event that of basis Inventry have update the our DB and generate the event. Payment have listen a event. Suppose Payment have fail. It's

Saga is two Types

1. Cherography
2. Orchertator

1:- Cherography :-

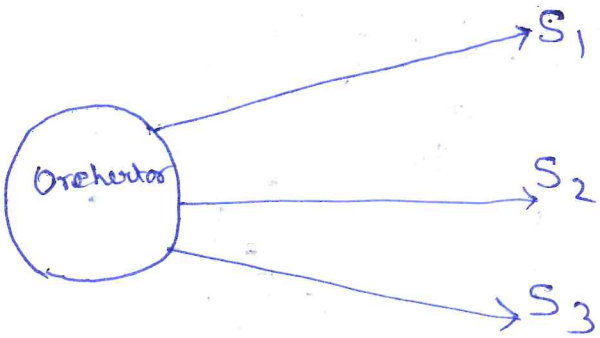


Concept && Coding

drawback

Cycle dependency

2:- Orchertator :-



* Orchertor is one who will call S_1. Orchertor based on replies on S_1 to call the S_2. Orchertor have again call S_3 based on S_2. Suppose S_3 will be fail. Orchertor have make sure that rollback [illegible]